УДК 004.312.2 : 519.713

М. П. Меньшиков (1 курс бакалавриата)

РАЗРАБОТКА И РЕАЛИЗАЦИЯ АЛГОРИТМА ФОРМАЛЬНОЙ ВЕРИФИКАЦИИ НА ОСНОВЕ МАТРИЧНЫХ ВЫЧИСЛЕНИЙ

ВВЕДЕНИЕ

Верификация – неотъемлемый процесс в проектировании электроники и критических систем, на который приходится до 60 – 80% общего цикла разработки. Однако только формальная верификация может гарантировать соответствие математической модели спецификации. Одна из разновидностей формальной верификации – проверка эквивалентности двух логических функций (Logic Equivalence Checking, LEC). При проектировании микропроцессоров LEC применяется при проверке соответствия RTL и Netlist, т. е. при проверке корректности логического синтеза [2].

Проверка эквивалентности – это NP-трудная задача и в общем случае за полиномиальное время не решается. Несмотря на это, существует ряд способов справиться с этой проблемой, которые активно применяют SAT-солверы. Например, представление логических схем в виде бинарных диаграмм решений (BDD) [4] или И-НЕ графов (And-Inverter Graph, AIG) [1]. Кроме того, существующие инструменты (например, программа ABC [5]) имеют богатый запас эвристик, за счёт которых на практике часто работают быстро, но в худшем случае работают за экспоненциальное время и заранее предсказать, сколько будет верифицироваться цифровая схема, инженеру довольно сложно, поэтому в некоторых случаях, если SAT-солвер не может быстро доказать эквивалентность, приходится вносить изменения в структуру дизайна (RTL рефакторинг), что увеличивает сроки выхода продукта.

Предлагается детерминированный алгоритм, который в худшем случае работает за экспоненциальное время, но с предсказуемым временем выполнения за счёт представления на диаграмме Эйлера множества не как переменной, а как дискретной функции в матричном виде дизъюнктивной нормальной формы (ДНФ). Алгоритм тестировался на комбинационных бенчмарках EPFL [3]. Кроме того, мной описаны границы применимости алгоритма, а именно в каких случаях происходит комбинаторный взрыв, что позволяет инженеру учитывать эти особенности на этапе проектирования цифровой схемы.

МАТЕМАТИЧЕСКОЕ ОПИСАНИЕ АЛГОРИТМА

Обратимся к понятию множества как к совокупности элементов, объединённых по какому-либо общему признаку [7]. Представим функцию в виде схемы Эйлера, на которой изображено 2 круга, каждый из которых обозначает дискретную функцию от многих[1] переменных.

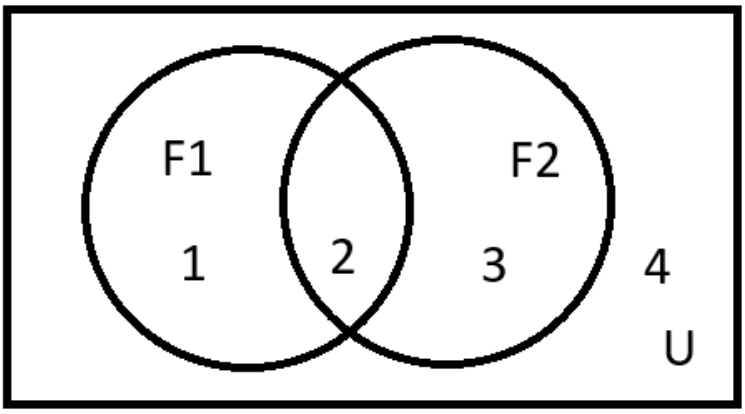


Рисунок 1 – диаграмма, описывающая 2 множества, содержащая 4 сектора: (1) – F1[2] И НЕ F2, (2) – F1 И F2, (3) – F2 И НЕ F1, (4) – НЕ F1 И НЕ F2, не зависимо от количества переменных в F1 и F2. U – универсальное множество

Результат выполнения каждой операции можно представить двумя способами:

1. Схема Эйлера, описывающая несколько множеств, каждое из которых состоит из одной переменной.
2. Диаграмма из не более чем 2-х множеств, описывающих дискретные функции, каждая из которых является результатом выполнения предыдущей операции.

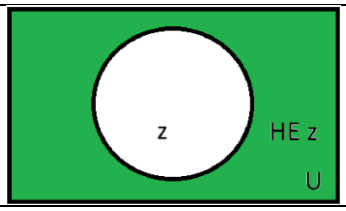
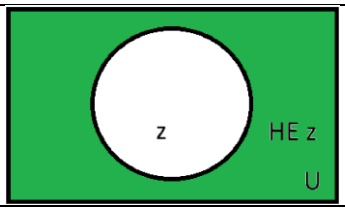
На примере ниже рассмотрим разницу между обоими способами представления.

Обозначим конъюнкцию как &, дизъюнкцию как |, инверсию – !

Над каждой операцией указан порядок выполнения.

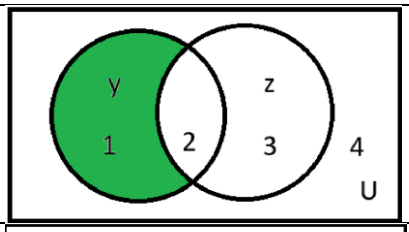
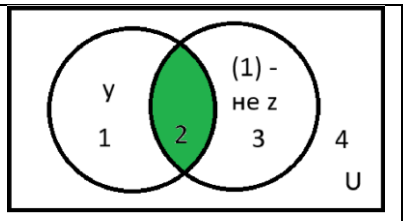
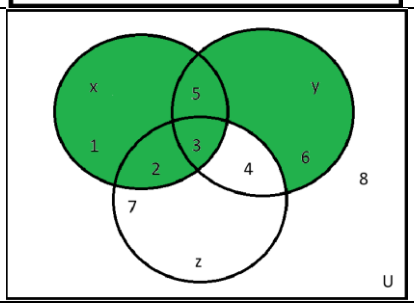
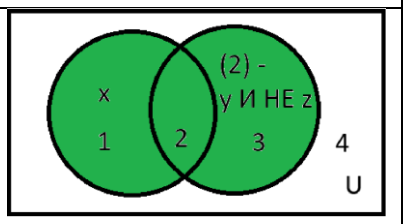
$F(x,y,z) \ = \ x \ |^{3} \ y \ \&^{2} \ !^{1} \ (z)$ (выражение 1)

Таблица 1

Порядок операции	Множество – одна переменная		Множество – дискретная функция	
	Описание области	Диаграмма	Описание области	Диаграмма
1	НЕ z		НЕ z	

[1] Многих – дискретная функция от сколь угодно большого количества переменных, в том числе одной переменной.

[2] Для понимания алгоритма введём классы P – просто переменная (например, x) и F – дискретная функция (например, НЕ x, x ИЛИ y, НЕ (x И y), x ИЛИ (y И НЕ z) и др.). Класс P принадлежит классу F, причём дискретная функция F, зависящая от одной переменной и не имеющая ни одной операции принадлежит классу P.

2	y И НЕ z		y И (1) (1): F(z)=!(z)	
3	x ИЛИ y И НЕ z		x ИЛИ (2) (2): F(y,z)=y&!(z)	

В классической диаграмме Эйлера (если множество – одна переменная) для 3-х переменных получается $2^3 = 8$ областей. Каждую область можно представить в виде элементарной конъюнкции. Составим таблицу истинности для (выражения 1).

Таблица 2

x	y	z	F	Соответствие области на диаграмме Эйлера
0	0	0	**0**	8
0	0	1	**0**	7
0	1	0	**1**	6
0	1	1	**0**	4
1	0	0	**1**	1
1	0	1	**1**	2
1	1	0	**1**	5
1	1	1	**1**	3

Таблица истинности содержит 8 строк, из которых 5 – истинные, 3 – ложные. Если истинные строки (описывают области 6, 1, 2, 5, 3) представить в виде элементарной конъюнкции, а сами строки связать операцией дизъюнкции, то получится запись в совершенной дизъюнктивной нормальной форме (СДНФ): (НЕ x И y И НЕ z) или (x И НЕ y И НЕ z) или (x И НЕ y И z) или (x И y И НЕ z) или (x И y И z). (1)

Таблица истинности – канонический способ записи выражения. Однако СДНФ содержит 5 элементарных конъюнкций, а минимальная ДНФ – 2: x ИЛИ (y И НЕ w).

Теперь обратимся к диаграмме, на которой множество обозначает дискретную функцию. Независимо от количества переменных диаграмма всегда содержит 1 или 2 круга. Однако, чтобы в виде логического выражения представить область, обозначенную на такой диаграмме, необходимо знать результат выполнения всех ведущих[3] операций (их может быть 0, 1 или 2), т. к. текущая операция k может ссылаться на предыдущие. В приведённом примере операция 2 ссылается на операцию 1: y И (1), а операция 3 – на операцию 2: x ИЛИ (2). Рассмотрим на примере (выражения 1).

[3] Ведущая операция – это операция, от которой зависит текущая операция. Например, если операция (3) представляется как x ИЛИ (2), то операция (2) – ведущая для операции (3), а операция (3) – зависимая от операции (2).

Таблица 3

<table>
<tr><th>k</th><th>Операция k</th><th>Количество ведущих операций</th><th>Результат ведущих операций</th><th>Результат операции k</th></tr>
<tr><td>1</td><td>НЕ z</td><td>0</td><td>-</td><td><table><tr><th>z</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table></td></tr>
<tr><td>2</td><td>y И (1)</td><td>1</td><td><table><tr><th>z</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>(1)</td><td><table><tr><th>y</th><th>z</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table></td></tr>
<tr><td>3</td><td>x ИЛИ (2)</td><td>1</td><td><table><tr><th>y</th><th>z</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>(2)</td><td><table><tr><th>x</th><th>y</th><th>z</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table></td></tr>
</table>

По количеству информации результат получается одинаковым – 8 строк, в промежуточных вычислениях – до 4 строк. С помощью таблицы истинности после выполнения каждой операции можно получить ДНФ, в частности СДНФ. Однако, если последовательно выполнять операции, представляя на диаграмме в виде кругов дискретные функции, являющиеся результатом ведущей операции, необязательно составлять и хранить классическую (полную) таблицу истинности, в которой количество строк растёт экспоненциально: 2^N строк, где N – количество переменных в выражении.

Будем хранить неполную таблицу истинности, а именно только истинные строки (F = 1), выполнив среди истинных строк поглощение и склеивание. Однако если в классической таблице истинности каждая переменная P занимает ровно 1 столбец и имеет 2 значения – 0 (при переводе в СДНФ означает НЕ P) и 1 (при переводе в СДНФ означает P), т. к. в СДНФ каждая переменная участвует в каждой элементарной конъюнкции, то в ДНФ, не являющейся СДНФ, в любой элементарной конъюнкции каждая переменная может иметь 3 варианта, представленных в таблице 4.

Таблица 4

Вариант			Описание
Десятичная	Двоичная		
0	0	0	В элементарной конъюнкции ДНФ нет ни переменной P, ни её отрицания НЕ P
1	0	1	В элементарной конъюнкции ДНФ есть переменная P
2	1	0	В элементарной конъюнкции ДНФ есть отрицание переменной НЕ P

Из таблицы 4 следует, что в неполной таблице истинности необходимо на каждую переменную отвести 2 столбца, первый будет обозначать состояние переменной P, второй – её отрицания НЕ P.

В классическом (полном) представлении таблицы истинности не может быть случая (0,0), в неполном – может. Остальные свойства неполной таблицы истинности наследуются от полной таблицы истинности. (2)

Учитывая выше сказанное, на примере (выражения 1) рассмотрим, как будет выглядеть итоговая таблица (таблица 5).

Таблица 5

x	НЕ x	y	НЕ y	z	НЕ z
1	0	0	0	0	0
0	0	1	0	0	1

Из (1) и (2) следует, что таблице 5 соответствует ДНФ: x ИЛИ (y И НЕ z). Назовём таблицу 5 матричным видом представления дизъюнктивной нормальной формы (3), а запись x ИЛИ (y И НЕ z) – явным видом.

Запись ДНФ состоит из дизъюнкций элементарных конъюнкций (далее – э. к.). Каждую э. к. можно представить в виде характеристического вектора из 2×N элементов, где N – количество переменных в выражении. В таблице 5 э. к. x можно представить в виде характеристического вектора σ (1,0,0,0,0,0), а э. к. y И НЕ z в виде характеристического вектора σ (0,0,1,0,0,1). Если k – это индекс переменной, то в столбце 2×k – состояние[4] переменной k, (2×k+1) – состояние её отрицания, $0 \leq k < N$. Заметим, что пара элементов в характеристическом векторе имеет 3 состояния, обозначающиеся как столбцы 2 и 3 в таблице 4: (0,0), (0,1), (1,0).

Объединение э. к. образует матрицу размером M×2*N, где M – количество э. к. в записи ДНФ, N – количество переменных в выражении. Каждая строка матрицы состоит из одного характеристического вектора σ. Чтобы перевести запись выражения из матричного вида в явный, достаточно между соседними строками поставить знак дизъюнкции ИЛИ. Это следует из (1) и (2).

Для того, чтобы выполнять операции в матричном виде, обратимся к свойствам булевых функций. Обозначим β как э. к., а $\sigma_n \sigma_k$ как поэлементное умножение[5] характеристических векторов.

1. **F(F1,F2) = F1 или F2**

ДНФ функции F1 можно представить в виде [β_0 или β_1 или … или β_{M-1}]. По свойствам (1) и (2) ей соответствует матрица, строками которой являются характеристические векторы [$\sigma_0, \sigma_1, \ldots, \sigma_{M-1}$].

---

[4] Состояние переменной k (равно как и её отрицания) равно либо 1, если переменная учитывается при переводе из матричного в явный вид ДНФ, либо 0, если переменная игнорируется.

[5] Поэлементное умножение характеристических векторов – это умножение каждого элемента σ_{ni} на элемент σ_{ki}, где i – элемент векторов σ_n и σ_k соответственно. Например, если σ_n = {1,0,1,0,1,0,0,0}, а σ_k = {0,0,0,0,1,0,1,0}, то $\sigma_n\sigma_k$ = {1,0,1,0,1,0,1,0}

Аналогично, ДНФ функции F2 можно представить в виде [β'_0 или β'_1 или … или $\beta'_{M'-1}$]. По свойствам (1) и (2) ей соответствует матрица, строками которой являются характеристические векторы [$\sigma'_0, \sigma'_1, \ldots, \sigma'_{M'-1}$].

Если выполнить операцию дизъюнкции между ДНФ функции F1 и ДНФ функции F2, то по свойству ассоциативности булевых функций результат F(F1,F2) = F1 или F2 = [β_0 или β_1 или … или β_{M-1}] или [β'_0 или β'_1 или … или $\beta'_{M'-1}$] = [β_0 или β_1 или … или β_{M-1} или β'_0 или β'_1 или … или $\beta'_{M'-1}$].

Из (1) и (2) следует, что результату F(F1,F2) соответствует матрица, строками которой являются характеристические векторы [$\sigma_0, \sigma_1, \ldots, \sigma_{M-1}, \sigma'_0, \sigma'_1, \ldots, \sigma'_{M'-1}$].

На рисунке 2 приведён пример выполнения операции дизъюнкции.

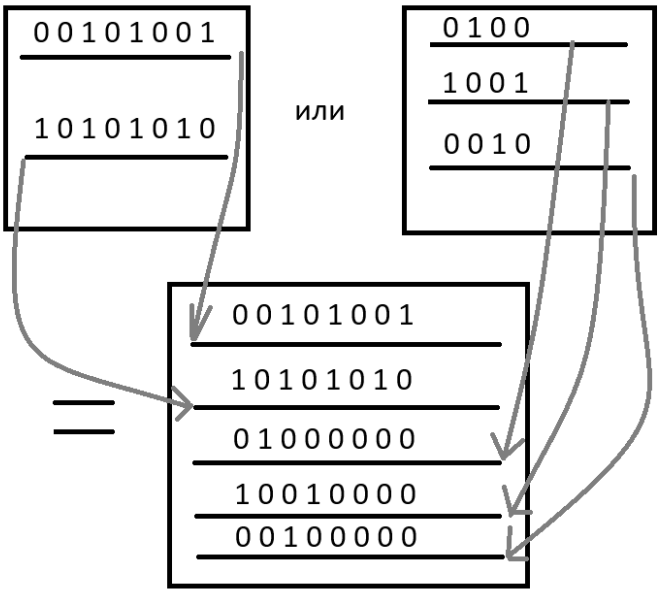


Рисунок 2 – пример операции дизъюнкции над ДНФ в матричном виде

2. F(F1,F2) = F1 и F2

ДНФ функции F1 можно представить в виде [β_0 или β_1 или … или β_{M-1}]. По свойствам (1) и (2) ей соответствует матрица, строками которой являются характеристические векторы [$\sigma_0, \sigma_1, \ldots, \sigma_{M-1}$].

Аналогично, ДНФ функции F2 можно представить в виде [β'_0 или β'_1 или … или $\beta'_{M'-1}$]. По свойствам (1) и (2) ей соответствует матрица, строками которой являются характеристические векторы [$\sigma'_0, \sigma'_1, \ldots, \sigma'_{M'-1}$].

Если выполнить операцию конъюнкции между ДНФ функции F1 и ДНФ функции F2, то по свойству дистрибутивности булевых функций результат F(F1,F2) = F1 и F2 = [β_0 или β_1 или … или β_{M-1}] и [β'_0 или β'_1 или … или $\beta'_{M'-1}$] = [$\beta_0\ \beta'_0$ или $\beta_0\ \beta'_1$ или … $\beta_0\ \beta'_{M'-1}$ или $\beta_1\ \beta'_0$ или $\beta_1\ \beta'_1$ или … $\beta_1\ \beta'_{M'-1}$ или … или $\beta_{M-1}\ \beta'_0$ или $\beta_{M-1}\ \beta'_1$ или … или $\beta_{M-1}\ \beta'_{M'-1}$].

Из (1) и (2) следует, что результату F(F1,F2) соответствует матрица, строками которой являются характеристические векторы [$\sigma_0\ \sigma'_0, \sigma_0\ \sigma'_1, \ldots, \sigma_0\ \sigma'_{M'-1}, \sigma_1\ \sigma'_0, \sigma_1\ \sigma'_1, \sigma_1\ \sigma'_{M'-1}, \ldots, \sigma_{M-1}\ \sigma'_0, \sigma_{M-1}\ \sigma'_1, \ldots, \sigma_{M-1}\ \sigma'_{M'-1}$].

На рисунке 3 приведён пример выполнения операции конъюнкции.

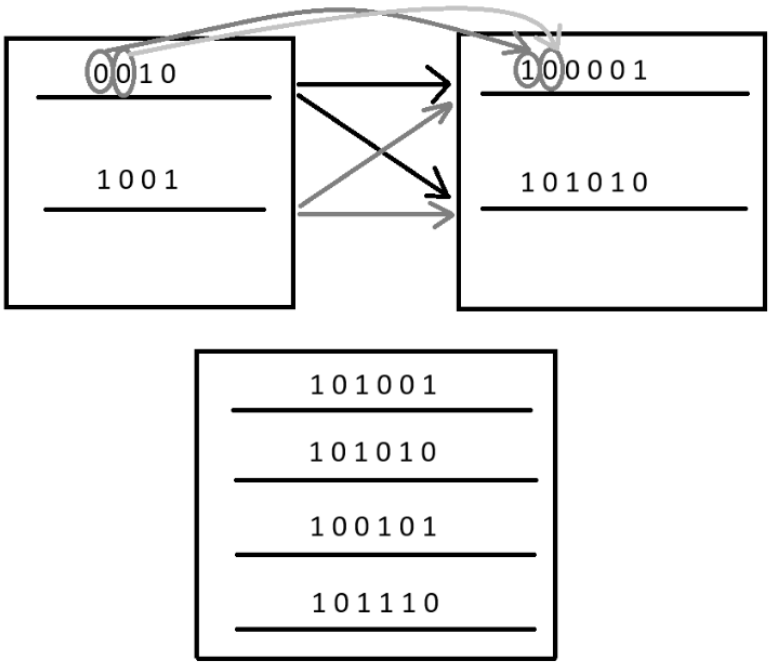


Рисунок 3 – пример операции конъюнкции в матричном виде

3. F(P) = не P

Операция отрицания переменной с индексом k – это запись в n-й характеристический вектор 1 в ячейку 2×k+1. Результату F(P) соответствует матрица, содержащая ровно одну строку n, являющуюся характеристическим вектором [σ_{n0}, …, $\sigma_{n(2k+1)}$, …, $\sigma_{n(2M-1)}$], причём $\sigma_{n(2k+1)}$ = 1, а остальные элементы равны 0.

Среди операций в матричном виде (п. 1 – 3) не выделяется инверсия дискретной функции (не F), допускается только инверсия одной переменной (не P). Докажем, что любую дискретную функцию можно представить в базисе *{не P, F1 и F2, F1 или F2}:*

Базис операций {не, и, или} является функционально полным. С помощью законов де Моргана [не (F1 и F2) = не F1 или не F2; не (F1 или F2) = не F1 и не F2)], выполняя их пока F1, F2 не станут P, можно представить любую дискретную функцию в базисе *{не P, F1 и F2, F1 или F2}.*

Обратимся к понятию Негативной нормальной формы (Negative Normal Form, NNF) [6]. Она обладает следующими важными для алгоритма свойствами:

1. Находится в базисе *{не P, F1 и F2, F1 или F2}*

2. Существует алгоритм приведения любой дискретной функции к NNF с помощью законов де Моргана [не (F1 и F2) = не F1 или не F2; не (F1 или F2) = не F1 и не F2] за линейное время O(N), где N – количество операций конъюнкции и дизъюнкции под операцией инверсии в исходной дискретной функции.

Учитывая выше сказанное, можно прийти к выводу, что сначала необходимо дискретную функцию привести к Негативной нормальной форме, а затем последовательно выполнить все операции в ней. Результатом выполнения каждой операции является матрица вида [3], т. е. любое промежуточное состояние описывается ДНФ в матричном виде.

На примере (выражения 1) последовательно выполним все операции в матричном виде в таблице 6.

Таблица 6

k	Операция k		ДНФ-1	ДНФ-2	ДНФ после операции k	
	Явный вид	Ссылочный	Матричный	Матричный	Матричный	Явный
1	НЕ z	НЕ z	z 1	-	z | не z 0 | 1	НЕ z
2	y И НЕ z	y И (1)	y 1	z | не z 0 | 1	y | z | не z 1 | 0 | 1	y И НЕ z
3	x ИЛИ (y И НЕ z)	x ИЛИ (2)	x 1	y | z | не z 1 | 0 | 1	x | y | z | не z 1 | 0 | 0 | 0 0 | 1 | 0 | 1	x ИЛИ (y И НЕ z)

Т. к. на входе (выражение 1) является ДНФ, то на выходе получается такое же выражение. В этом (частном) случае ДНФ минимальна по количеству э. к. и не требует применения законов поглощения и склеивания ни на одной из трёх операций.

Однако в общем случае сразу после выполнения каждой операции необходимо минимизировать ДНФ. В матричной форме удобно применить следующие законы:

1. Идемпотентность: X и X = X
2. Противоречие: X и (не X) = 0
3. Поглощение: X или (X и A) = X

На примере выражения 2 рассмотрим применение законов поглощения. Составим таблицу 7.

$$F(x,y,z,w) = \big(x \mid y \,\&\, !\,(w)\big) \,\&\, (!\,(y \,\&\, z) \mid x)$$

Т. к. в (выражении 2) присутствует !(F): !(y & z), выражение 2 не является негативной нормальной формой, поэтому перед выполнением операций приведём его к NNF.

$$F(x,y,z,w) = \big(x \mid^3 y \,\&^2\, !^1\,(w)\big) \,\&^8 (\,!^4\,(y) \mid^6 !^5\,(z) \mid^7 x)$$

Таблица 7

k	Операция k		ДНФ-1	ДНФ-2	ДНФ после операции k	
	Явный вид	Ссылочный	Матричный	Матричный	Матричный	Явный
1	НЕ w	НЕ w	w 1	-	w | не w 0 | 1	НЕ w
2	y И НЕ w	y И (1)	y 1	w | не w 0 | 1	y | w | не w 1 | 0 | 1	y И НЕ w
3	x ИЛИ (y И НЕ w)	x ИЛИ (2)	x 1	y | w | не w 1 | 0 | 1	x | y | w | не w 1 | 0 | 0 | 0 0 | 1 | 0 | 1	x ИЛИ (y И НЕ w)
4	НЕ y	НЕ y	y 1	-	y | не y 0 | 1	НЕ y
5	НЕ z	НЕ z	z 1	-	z | не z 0 | 1	НЕ z
6	НЕ y ИЛИ НЕ z	(4) ИЛИ (5)	y | не y 0 | 1	z | не z 0 | 1	y | не y | z | не z 0 | 1 | 0 | 0 0 | 0 | 0 | 1	НЕ y ИЛИ НЕ z

<table>
<tr>
<td>7</td>
<td>НЕ y ИЛИ НЕ z ИЛИ x</td>
<td>(6) ИЛИ x</td>
<td>
<table>
<tr><td>y</td><td>не y</td><td>z</td><td>не z</td></tr>
<tr><td>0</td><td>1</td><td>0</td><td>0</td></tr>
<tr><td>0</td><td>0</td><td>0</td><td>1</td></tr>
</table>
</td>
<td>
<table>
<tr><td>x</td></tr>
<tr><td>1</td></tr>
</table>
</td>
<td>
<table>
<tr><td>x</td><td>y</td><td>не y</td><td>z</td><td>не z</td></tr>
<tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr>
<tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr>
<tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr>
</table>
</td>
<td>НЕ y ИЛИ НЕ z ИЛИ x</td>
</tr>
<tr>
<td>8</td>
<td>(x ИЛИ y И НЕ w) И (НЕ y ИЛИ НЕ z ИЛИ x)</td>
<td>(3) И (7)</td>
<td>
<table>
<tr><td>x</td><td>y</td><td>w</td><td>не w</td></tr>
<tr><td>1</td><td>0</td><td>0</td><td>0</td></tr>
<tr><td>0</td><td>1</td><td>0</td><td>1</td></tr>
</table>
</td>
<td>
<table>
<tr><td>x</td><td>y</td><td>не y</td><td>z</td><td>не z</td></tr>
<tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr>
<tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr>
<tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr>
</table>
</td>
<td>
<table>
<tr><td>x</td><td>y</td><td>не y</td><td>z</td><td>не z</td><td>w</td><td>не w</td></tr>
<tr><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr>
<tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td></tr>
<tr><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td></tr>
<tr><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr>
</table>
Порядок минимизации:
1. Строка 3, идемпотентность: x&x=x. Строка 3 остаётся.
2. Строка 4, противоречие: y&!(y)=0. Строка 4 удаляется.
3. Строки 1, 2, 6, поглощение со строкой 3: x | (x&!(y)) = x, x | (x&!(z))=x, x | (x&y&!(w))=x Строка 3 остаётся, 1, 2, 6 – удаляются.
На выходе сохранились строки 3 и 5
</td>
<td>x ИЛИ (y И НЕ z И НЕ w)</td>
</tr>
</table>

Итак, описанный способ позволяет представить любую дискретную функцию в ДНФ. Этот способ представления удобен тем, что даёт возможность быстро[6] проверить функцию на выполнимость (SAT). Заметим, что для того, чтобы проверить эквивалентность двух логических выражений, достаточно выполнить операцию $F(F1, F2) = F1\ XOR\ F2 =$ (не $F1$ и $F2$) или ($F1$ и не $F2$) и если F(F1,F2) выполнима, значит F1≡F2.

На выходе алгоритм выводит ДНФ расхождения между функциями F1 и F2, а именно ДНФ функции F(F1,F2). Если в ДНФ функции F(F1,F2) нет ни одной э. к., значит она выполнима, а следовательно F1≡F2.

[6] Проверка ДНФ на выполнимость возможна за линейное время. Для этого хотя бы одна э. к. не должна обращаться в 0. Т. к. э. к. обращается в 0 только при наличии P&!(P)=0 и такие э. к. в описанном алгоритме уничтожаются сразу после возникновения в ходе построения ДНФ, приходим к выводу: если после составления ДНФ нет ни одной э. к., а именно ни одного характеристического вектора в матрице (3), значит функция выполнима.

ГРАНИЦЫ ПРИМЕНИМОСТИ

На диаграмме Эйлера N кругов[7] образуют 2^N секторов, а значит любая ДНФ содержит не более 2^N импликант[8]. Минимальная ДНФ в худшем случае имеет 2^{N-1} импликант. Так как в ходе алгоритма на каждой операции с целью минимизации применяется только закон поглощения, не гарантируется, что ДНФ всегда будет минимальной. Следовательно, количество импликант M в худшем случае удовлетворяет неравенству $2^{N-1} \leq M \leq 2^N$. Худший случай возникает, если функция чётна[9] или нечётна. В таком случае невозможно объединить сектора на диаграмме Эйлера (выполнить поглощение и склеивание). Пример функции от 3 переменных, имеющей на диаграмме Эйлера $2^3 = 8$ секторов и имеющей минимальную ДНФ из $2^{3-1} = 2^2 = 4$ импликант, представлен на рисунке 4.

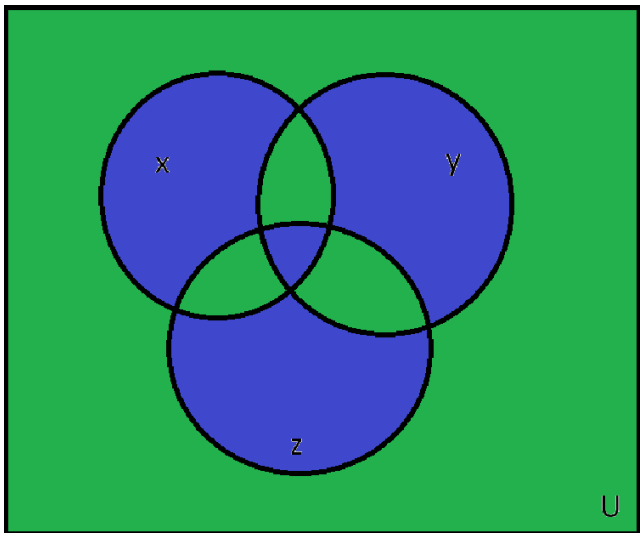


Рисунок 4 – диаграмма Эйлера; зелёная область – чётная функция (4 сектора; 4 импликанты в минимальной ДНФ), синяя область – нечётная функция (4 сектора; 4 импликанты в минимальной ДНФ)

Результат работы алгоритма – ДНФ расхождения дискретных функций F1 и F2, а именно поиск функции $F(F1, F2) = F1\, XOR\, F2$. Будем считать, что если их ДНФ расхождения содержит более $3N$ импликант (где N – количество переменных), то размер ДНФ функции F(F1,F2) значительный; если количество импликант находится в диапазоне $(0; 3N]$ – размер ДНФ функции F(F1,F2) незначительный; если количество импликант равно 0, то функции F1 и F2 эквивалентны.

Если размер ДНФ расхождения значительный, то программа не выдаст ответа из-за экспоненциально больших выходных данных. Отсюда можно сделать вывод, что алгоритм применим не к любым функциям F1 и F2, а только если F1≡F2 либо ДНФ функции F(F1,F2) имеет незначительный размер. Если это не так, программа будет работать экспоненциальное время. Задача LEC решается при проверке эквивалентности цифровой схемы до логического синтеза и после него. В абсолютном большинстве случаев в задачах логического синтеза F1 и F2 эквивалентны или размер их ДНФ расхождения незначительный [2], поэтому описанный в статье алгоритм имеет применение в задаче LEC. Т. к. инженер не уверен[10] на момент подачи F1 и F2 на вход программы, что F1 и F2 эквивалентны, то если за приемлемое время программа

---

[7] Каждый круг соответствует одной переменной

[8] Импликанта – это ненулевая выполнимая э. к.

[9] Чётная функция выполняется, когда количество единиц на входе чётно, т. е. в каждой чётной строке таблицы истинности. Например, F(x,y) = x XOR y

[10] Если бы это было не так, задача LEC потеряла бы смысл

не смогла завершить работу – значит одна из причин, что F1 и F2 расходятся значительно, размер выхода (ДНФ расхождения) будет экспоненциально большим и логический синтез был выполнен неправильно.

Важная характеристика работы алгоритма – зависимость верхней границы количества импликант (строк в матрице вида (3)) в промежуточных вычислениях от количества импликант в ДНФ расхождения (в выходных данных).

На рисунке 5 представлен график зависимости количества импликант на каждой операции. На вход программе подавался бенчмарк adder EPFL (выход 564) [3].

Рисунок 5 – график зависимости количества импликант на каждой операции. По оси абсцисс – № операции, по оси ординат – количество импликант.

Назовём переменную критической, если она существует не менее, чем в 90% импликант ДНФ. Анализируя график (рисунок 5), можно заметить, что возможны 3 ситуации, при которых за одну операцию значительно[11] уменьшается количество импликант в ДНФ:

1. Положительный ответ на вопрос: “Существует ли с ДНФ, полученной в результате текущей операции такая конъюнкция в предстоящих операциях, что её ДНФ содержит хотя бы один импликант из отрицания критической переменной и не содержит ни одного импликанта, не содержащего отрицания критической переменной?” Рассмотрим на примере таблицы 8.

Таблица 8

Результат текущей операции				Результат предстоящей операции				Итоговый результат							
x	y	z	w	!x	!y	!z	!w	x	!x	y	!y	z	!z	w	!w

[11] Значительным сокращением количества импликант за одну операцию будем считать уменьшение их количества не менее, чем в 10 раз. Нередко их количество сокращается до 0.

1		1	1
1	1		1
1	1	1	

1	1	1	
1	1		1
1		1	1

1	1		1	1	1	1	
1	1		1	1		1	1
1	1			1	1	1	1
1	1	1	1		1	1	
1	1	1	1			1	1
1	1	1			1	1	1
1	1	1	1	1	1		
1	1	1	1	1			1
1	1	1		1	1		1

Переменная x – критическая, т. к. присутствует в 3 из 3 импликант (100%) в результате текущей операции. Соответственно, конъюнкция с !x по закону противоречия уничтожает импликант. В таблице 8 итоговый результат содержит 0 импликант, хотя до операции конъюнкции в сумме было 6 импликант, что считается значительным уменьшением.

2. Положительный ответ на вопрос: “Существует ли с ДНФ, полученной в результате текущей операции такая дизъюнкция в предстоящих операциях, что её ДНФ содержит хотя бы один импликант, состоящий только из критической переменной и не содержащий других переменных, в т. ч. отрицания критической переменной?” Рассмотрим на примере таблицы 9.

Таблица 9

Результат текущей операции				Результат предстоящей операции				Итоговый результат			
x	y	z	w	x	y	z	w	x	y	z	w
1		1	1	1				1		1	1
1	1		1					1	1		1
1	1	1						1	1	1	
								1			

В приведённом в таблице 9 примере переменная x – критическая, т. к. встречается в 3 из 3 импликантах (100%). Однако по закону поглощения дизъюнкция с x сохраняет только одну импликанту – x. Этот случай масштабируется на сколь угодно большое количество импликант, содержащих x в результате текущей операции.

3. Положительный ответ на вопрос: “Существует ли с ДНФ, полученной в результате текущей операции конъюнкция с нулём?”
   Конъюнкция с нулём обращает в ноль ДНФ любого размера.

Получить ответы на 3 описанных вопроса в общем случае возможно только за экспоненциальное время. Однако если за фиксированное[12] количество операций, стоящих после текущей, подтвердится один из трёх фактов, то текущую операцию можно не выполнять, а импликанты не записывать.

Т. к. размер ДНФ значительный, если превышает $3N$ импликант, а значительное уменьшение количества импликант за одну операцию – в 10 раз, то следует отвести на хранение в промежуточной ДНФ $30N$ импликант.

Отсюда следует, что если на каждой операции, представление результата которой в ДНФ превышает $30N$ импликант, возможно за фиксированное число итераций доказать, что предстоящая операция выполняет один из трёх случаев значительного уменьшения количества импликант, то алгоритм гарантированно отработает за приемлемое время. (4)

[12] На текущем этапе исследования не определено точное количество операций, за которое алгоритм должен доказать, что выполнять текущую операцию не требуется.

Заметим, что задача эквивалентности в логическом синтезе при условии, что F1 и F2 эквивалентны или размер их ДНФ расхождения незначительный – частный случай ограничения (4), т. к. если за фиксированное число итераций невозможно доказать, что предстоящая операция выполняет один из трёх случаев значительного уменьшения количества импликант, то либо такую операцию не удалось найти из-за NP-полноты, либо её не существует – что свидетельствует о том, что ДНФ функции F(F1,F2) имеет значительный размер.

Итак, описанный в статье алгоритм применим и отработает за приемлемое время, если выполняется следующее условие:

На каждой операции, представление результата которой в ДНФ превышает 30N импликант (где N – количество переменных на входе), возможно за фиксированное число итераций доказать, что предстоящая операция является хотя бы одим из трёх случаев значительного уменьшения количества импликант:

1. *Положительный ответ на вопрос: “Существует ли с ДНФ, полученной в результате текущей операции такая конъюнкция в предстоящих операциях, что её ДНФ содержит хотя бы один импликант из отрицания критической переменной и не содержит ни одного импликанта, не содержащего отрицания критической переменной?”*
2. *Положительный ответ на вопрос: “Существует ли с ДНФ, полученной в результате текущей операции такая дизъюнкция в предстоящих операциях, что её ДНФ содержит хотя бы один импликант, состоящий только из критической переменной и не содержащий других переменных, в т. ч. отрицания критической переменной?”*
3. *Положительный ответ на вопрос: “Существует ли с ДНФ, полученной в результате текущей операции конъюнкция с нулём?”*

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Armin Biere, Johannes Kepler University, The AIGER And-Inverter Graph (AIG). [Электронный ресурс] Режим доступа: fmv.jku.at/aiger/FORMAT.aiger (дата обращения: 16.03.2026)

2. Хабр. Разработка и производство современных ASIC/SoC глазами тополога. [Электронный ресурс] Режим доступа: https://habr.com/ru/companies/yadro/articles/883322 (дата обращения: 02.03.2026)

3. GitHub. The EPFL Combinational Benchmark Suite. [Электронный ресурс] Режим доступа: https://github.com/lsils/benchmarks (дата обращения: 29.03.2026)

4. Wikipedia. Binary decision diagram. [Электронный ресурс] Режим доступа: https://en.wikipedia.org/wiki/Binary_decision_diagram (дата обращения: 20.02.2026)

5. GitHub. AIGER And-Inverter-Graph Library. [Электронный ресурс] Режим доступа: https://github.com/arminbiere/aiger (дата обращения: 17.03.2026)

6. Wikipedia. Negative normal form. [Электронный ресурс] Режим доступа: https://en.wikipedia.org/wiki/Negation_normal_form (дата обращения: 25.02.2026)

7. Хаггарти Р. Дискретная математика для программистов : учебное пособие / Р. Хаггарти ; пер. с англ. С. А. Кулешова. – 2-е изд., испр. – Москва : Техносфера, 2012. – 400 с. – ISBN 978-5-94836-303-5. – Раздел 3.1. – С. 44 – 51.

8. The-formal-verifier-LEC : разработка и реализация алгоритма формальной верификации на основе матричных вычислений : исходный код на языке C++ / М. П. Меньшиков репозиторий на GitHub. – URL: https://github.com/Matvey-hue/The-formal-verifier-LEC (дата обращения: 12.04.2026).